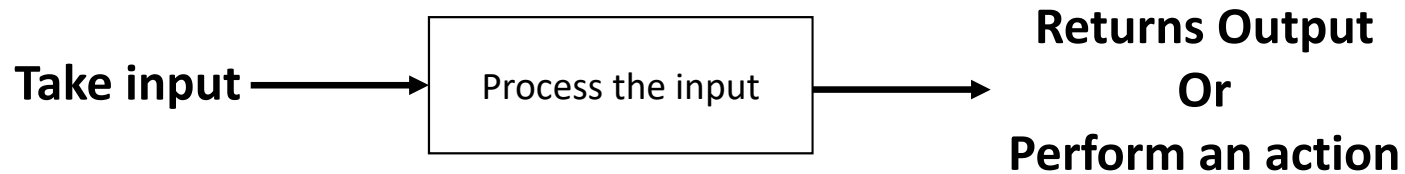


Function

Functions are the reusable blocks of code that perform a specific task.



Why are the functions important:

- Avoid repeating code
- Make the program easier to read
- Break big problems into smaller parts
- Fix or update the code more easily

Functions Vs. Procedure

In C programming:

No return	Procedure	Void
Return	Function	Int/float

In Python:

Python does not officially differentiate between functions and procedures

Function

Types of Functions

➤ Library functions

Categories	Functions
Input / Output	print(), input()
Type conversion	int(), float(), str(), bool(), list(), tuple(), set()
Mathematical	sum(), max(), min(), abs(), round(), pow()
Sequence operation	len(), sorted(), reversed(), range(), enumerate(), zip()
Logical	all(), any()
Object information	type(), id(), help()

Function

User-defined functions

➤ Syntax:

```
def functionname(parameters):  
    statements  
  
functionname(argument)
```

- **Keyword def marks the start of the function header.**
- **Parameters through which we pass a value to a function. They are optional.**
- **An optional return.**

```
: def square(data):  
    y=data*data  
    return(y)  
  
print("enter the value of x")  
x=int(input(">"))  
z=square(x)  
print(z)
```

```
enter the value of x  
>3  
9
```

Function

Functions parameters and arguments

- A **parameter** is the variable listed inside the parentheses in the function definition.
- An **argument** is the actual value that is sent to the function when it is called.

```
def my_function(name): # name is a parameter
    print("Hello", name)

my_function("Ram") # "Ram" is an argument

Hello Ram
```

Types of parameters

- Positional Parameters
- Keyword parameters
- Default parameters
- Variable length parameters
 - Variable length positional parameters (*args)
 - Variable length keyword parameters (*kwargs)

Function

Positional parameters

- **Positional parameters** are parameters that receive values based on the **position (order)** of the arguments passed during a function call.
- The order of the arguments **must match** the order of the parameters in the function definition.

Keyword parameters

- **Keyword parameters** allow arguments to be passed to a function by **specifying the parameter name**.
- Since the parameter names are used, the **order of the arguments does not matter**.

Default parameters

- A **default parameter** is a parameter that is assigned a **default value** in the function definition.
- If no argument is passed for that parameter when the function is called, Python uses the default value.

```
def student(name, age):  
    print("Name:", name)  
    print("Age:", age)  
  
student("Riya", 18)
```

Name: Riya
Age: 18

```
def student(name, age):  
    print("Name:", name)  
    print("Age:", age)  
  
student(age=18, name="Riya")
```

Name: Riya
Age: 18

```
def greet(name="Student"):  
    print("Hello", name)  
  
greet()  
greet("Riya")
```

Hello Student
Hello Riya

Function

Variable length positional parameters (*args)

- A variable-length positional parameter allows a function to accept **any number of positional arguments**.
- The arguments are collected into a **tuple** using the *args syntax.

```
def add_numbers(*args):  
    total = 0  
    for num in args:  
        total += num  
    return total  
add_numbers(1,2)  
#add_numbers(1,2,3)
```

Variable length keyword parameters (*kwargs)

- A variable-length keyword parameter allows a function to accept **any number of keyword arguments**.
- These arguments are collected into a **dictionary** using the **kwargs syntax.

```
def student(**kwargs):  
    for key,value in kwargs.items():  
        print(key,value)
```

```
student(name="Rahul",age=20)
```

```
name Rahul  
age 20
```

```
student(name="Ram", age=30, salary=25000)
```

```
name Ram  
age 30  
salary 25000
```

Order

Positional --- *args---- Default-----Keyword--- **kwargs

Nested function

Function

Variable scope in functions

- **Local Variable:** accessible only inside the functional block where it is declared
- **Global variable:** A variable that is accessible throughout the whole program using the global keyword
- **Non Local Variable:** accessible in nesting of functions, using the nonlocal keyword

```
def xyz():  
    a=2  
    print(a)  
xyz()
```

Function

Global Variable: A variable that is accessible throughout the whole program
using the global keyword

```
def outer():  
    global a  
    a=10  
    def inner():  
        a=5  
        print(a)  
    inner()  
    a=15  
    print(a)  
outer()  
c=a*a  
print(c)
```

5
15
225

```
def outer():  
    global a  
    a=10  
    def inner():  
        a=5  
        print(a)  
    inner()  
    print(a)  
outer()  
c=a*a  
print(c)
```

5
10
100

```
: def outer():  
    a=10  
    def inner():  
        global a  
        a=5  
        print(a)  
    inner()  
    a=15  
    print(a)  
outer()  
c=a*a  
print(c)
```

5
15
25

Function

Non Local Variable: accessible in nesting of functions, using the nonlocal keyword

```
: def outer():  
    a = 10  
    def inner():  
        a = 20    # Creates a new local variable  
        print("Inner:", a)  
    inner()  
    print("Outer:", a)  
outer()
```

Inner: 20
Outer: 10

```
: def outer():  
    a = 10  
    def inner():  
        nonlocal a  
        a = 20  
        print("Inner:", a)  
    inner()  
    print("Outer:", a)  
outer()
```

Inner: 20
Outer: 20

```
def outer():  
    x = 1  
    def middle():  
        x = 2  
        def inner():  
            nonlocal x  
            x = 3  
        inner()  
        print("Middle:", x)  
    middle()  
    print("Outer:", x)  
outer()
```

Middle: 3
Outer: 1

Lambda Function (Anonymous Function)

- A lambda function is a small anonymous function.
- The power of lambda is better shown when you use them as an anonymous function inside another function.
- A lambda function can take any number of arguments, but can only have one expression.
- The expression is executed and the result is returned.

Syntax lambda arguments : expression

Example Square= lambda X: X*X

Lambda Function (Anonymous Function)

Multiple arguments:

Sum= lambda a,b: a+b

Without assignment:

Print((lambda X: X*X),(5))

Print((lambda a,b:a+b),(5,6))

Limitations:

- **Only one expression**
- **No loops and conditions (except conditional expression)**

Conditional lambda example:

max_num=lambda a,b: a if a>b else b

Only ternary

Recursive function

Recursive Program

WAP to find the factorial of a number n

- Python also accepts function recursion, which means a defined function can call itself.
- Recursion is a common mathematical and programming concept. It means that a function calls itself. This has the benefit of allowing you to loop through data to reach a result.
- Every recursive function has:
 - Base Case: Stop the recursion
 - Recursive Case: Function call itself

```
: def factorial(n):  
    fact =1  
    for i in range (1,n+1):  
        fact=fact*i  
    return fact  
  
number= int(input("Enter the number"))  
factor = factorial(number)  
  
print("The factorial of", number, " numbers" ,factor)
```

```
: # with recursive  
def factorial(n):  
    if n==1:  
        return 1  
    else:  
        print(n)  
        return n*factorial(n-1)  
  
number= int(input("Enter the number"))  
factor = factorial(number)  
  
print("The factorial of", number, " numbers", factor)
```

Modules

Modules

Math modules

```
: import math  
print(math.pi)  
  
: print(math.e) # euler's number  
  
: print(math.sqrt(2))
```

Modules

Random module

generate random numbers and perform random operations

```
: import random  
  
: print(random.random()) #random value between 0 to 1  
  
: print(random.randint(1,10)) # random integer value between a and b  
  
: print(random.uniform(1,10)) # random float value between a and b
```

uniform

10 random numbers

```
random_1=[]  
for i in range(10):  
    a=random.random()  
    random_1.append(a)  
print(random_1)
```

Modules

Datetime module

It is used to work with dates and times.

```
: from datetime import datetime, date, time, timedelta
```

```
: now = datetime.now()  
print(now)
```

```
: # Current date only  
today = date.today()  
print(today)
```

```
: # Currenttime  
current_time = datetime.now().time()  
print(current_time)
```

```
: # Pause in python  
import time  
print("Wait for 5 seconds...")  
time.sleep(5)  
print("OK")
```

Exercise- 8

Write a program to accept two integer values from the user and return their product. If the product is greater than 1000, then return their sum.

```
def multiplication_or_sum(num1, num2):  
    product = num1 * num2  
    if(product < 1000):  
        return product  
    else:  
        return num1 + num2  
number1 = int(input("Enter first number "))  
number2 = int(input("Enter second number"))  
result = multiplication_or_sum(number1, number2)  
print("The result is", result)
```

```
Enter first number 12  
Enter second number 10  
The result is 120
```

Recursive Program

Linear Search

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1  # target not found

nums = [10, 25, 7, 18, 30]
result = linear_search(nums, 7)

if result != -1:
    print("The target number is at {} index value".format(result))
else:
    print("The target number is not available")
```

The target number is at 2 index value

```
: def linear_search(arr, target, index=0):
    # Base case: reached end of list
    if index == len(arr):
        return -1

    # If element is found
    if arr[index] == target:
        return index

    # Recursive case: check next element
    return linear_search(arr, target, index + 1)

nums = [10, 25, 7, 18, 30]

result = linear_search(nums, 7)
if result != -1:
    print("The target number is at {} index value".format(result))
else:
    print("The target number is not available")
```

Recursive Program

Linear Search

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1  # target not found

nums = [10, 25, 7, 18, 30]
result = linear_search(nums, 7)

if result != -1:
    print("The target number is at {} index value".format(result))
else:
    print("The target number is not available")
```

The target number is at 2 index value

```
: def linear_search(arr, target, index=0):
    # Base case: reached end of list
    if index == len(arr):
        return -1

    # If element is found
    if arr[index] == target:
        return index

    # Recursive case: check next element
    return linear_search(arr, target, index + 1)

nums = [10, 25, 7, 18, 30]

result = linear_search(nums, 7)
if result != -1:
    print("The target number is at {} index value".format(result))
else:
    print("The target number is not available")
```

filter()

Select elements from an iterable that satisfy a condition.

Syntax: filter(function, iterable)

```
# find the even number
```

```
a=[1,2,3,4,5,6]
def even(x):
    if x%2==0:
        return x
z=filter(even,a)
print(list(z))
```

```
[2, 4, 6]
```

```
# with lambda function
```

```
a=[1,2,3,4,5,6]
z=filter(lambda x:x%2==0,a)
print(list(z))
```

```
[2, 4, 6]
```

Built-in Functions

all(): checks whether **all elements** in an iterable are **True**.

Syntax: `all(iterable)`

```
x = [1, 2, 3, 4]

print(all(x))
```

True

```
x = [1, 2, 0, 4]

print(all(x))
```

False

any(): checks whether **at least one element** in an iterable is **True**.

Syntax: `any(iterable)`

```
x = [0, 0, 5, 0]

print(any(x))
```

True

```
x = [0, 0, 0]

print(any(x))
```

False